

I'll drop the SVG/Figma build and give you the complete deep-dive write-up instead, in chat, covering everything discussed across this whole conversation — physics, math, exact component names, logic, algorithms, and flow, end to end.

Environmental Intelligence Network — Complete Technical Reference

PART 1: THE CORE PHILOSOPHY

1.1 Detector vs. Predictor — the foundational distinction

A **detector** is a system that reacts once a condition is already true. A water sensor that fires an alarm when the river crosses a fixed height (say, 3 meters) is a detector. Its fundamental flaw: by the time water is at 3 meters, the flood is already arriving. People downstream get minutes of warning, sometimes none.

A **predictor** is a system that identifies the *pattern that precedes* a dangerous condition, before that condition exists. This requires:

- Multiple sensing points spread across a geographic area (not one sensor)
- A memory of what "normal" looks like at each point, built from historical data
- A method to detect deviation from that normal, and to project forward: given this deviation, when will its effects reach a different point?

The entire architecture below exists to make this second thing possible, not the first.

1.2 Why fixed thresholds are wrong, physically

A river's baseline behavior is a function of: catchment area, seasonal rainfall, snowmelt contribution (relevant in Himalayan contexts), channel geometry, and even time of day for glacier-fed streams. There is no universal number for "dangerous." A water level or turbulence value that is completely routine on a wide monsoon-fed lowland river would represent a major event on a narrow, steep mountain torrent, and vice versa.

This means: **any alerting logic that uses an absolute, hardcoded threshold value will either produce constant false alarms on naturally turbulent rivers, or fail to trigger at all on naturally calm ones that are actually in danger.**

The mathematical fix is to make "normal" a *learned, per-site, time-varying quantity* rather than a constant. This is developed fully in Part 5.

PART 2: SENSING — WHAT EACH NODE MEASURES AND HOW

2.1 Why we rejected several sensing approaches, in order

Rejected: MPU6050 accelerometer for seismic/flood vibration sensing as a primary signal.

Physics reason: the MPU6050 is a MEMS accelerometer with a noise density around 400 micro-g per root-Hz ($\mu\text{g}/\sqrt{\text{Hz}}$), and typical resolution of about 0.06 mg per LSB at the $\pm 2\text{g}$ range setting. Real-world earthquake or flood-adjacent ground vibration signals that matter — especially from moderate or distant events — often sit in the range of single-digit to tens of milli-g. That's within the same order of magnitude as the sensor's own noise floor, meaning the signal gets buried in noise. Additionally, the MPU6050 has no built-in temperature compensation, and MEMS accelerometers drift thermally — over the timescales relevant to slow-building hazards (hours), this drift alone can exceed the signal you're trying to measure. Proper seismic-grade instruments (geophones, or accelerometers like the ADXL355 with noise density near $25 \mu\text{g}/\sqrt{\text{Hz}}$) are roughly an order of magnitude quieter.

Rejected: ground vibration sensing at 50-100m from a riverbank to detect flow-based vibration.

Physics reason: soil is a lossy transmission medium. Vibrational energy from turbulent water flow attenuates and disperses as it travels through ground — high-frequency components attenuate fastest. At 50-100 meters, whatever signal originated at the riverbank is heavily attenuated and buried under everyday ambient ground noise (wind loading on structures, vehicle traffic, animals, thermal expansion/contraction of the sensor's own mount). This is true even with seismic-grade equipment; the physical signal path itself is too weak at that distance for this specific technique, independent of sensor quality.

Rejected: ultrasonic distance sensors (e.g., HC-SR04-class) for water level at any meaningful mounting height.

Physics reason: cheap ultrasonic time-of-flight sensors have a practical reliable range of roughly 4-5 meters, with accuracy degrading substantially beyond 2-3 meters. Given that Himalayan flood surges can rise 5-7 floors (roughly 15-25 meters) above baseline river level, any sensor mounted high enough to physically survive such a surge is now far outside ultrasonic's usable range. Ultrasonic is also sensitive to wind, temperature, and humidity gradients in the air column between sensor and target — conditions that are actively adverse during the storm events you most need reliable readings from.

Rejected: standard industrial radar level sensors at 15-25m range in open river valleys.

Reason: while industrial non-contact radar (used in tanks, silos) is often rated for 15-30+ meters, that rating assumes a controlled, contained environment. An open-air mountain river valley introduces wind, spray, surface turbulence, and highly variable target geometry (the water surface itself is chaotic during a flood) that degrades real-world performance well below the datasheet rating for a contained vessel. This is a real, unresolved gap flagged for direct consultation with domain-expert mentors (hydrology/ISRO-adjacent), since actual operational Himalayan river gauge stations (run by India's Central Water Commission) have solved this

class of problem and are the correct reference to consult rather than reverse-engineering it from consumer sensor datasheets.

Adopted, with caveats: camera-based vision from a stable elevated position.

Reasoning: unlike echo-based sensing (ultrasonic, radar), optical sensing degrades much more gracefully with distance — resolvability is a function of focal length and sensor resolution, not signal attenuation through a medium. This is why the final design uses a camera mounted on **stable high ground** (a hillside or riverbank slope, not a pole standing in the flood's path), angled downward at roughly 30-40 degrees, giving a single frame that captures both the river below and the forest/slope above.

2.2 The optics problem, in detail

At a real-world distance of approximately 20-30 meters (10 floors, per your stated deployment plan), the governing constraint is **resolution-on-target**: how many pixels of the sensor actually correspond to the feature you need to measure.

Standard hobbyist camera modules (Sony/OmniVision sensors like the **OV2640** or the base **OV5647**) ship with wide-angle, fixed-focus lenses intended for close-range framing (a few meters). At 30 meters through a wide lens, a target object occupies a very small angular fraction of the frame — potentially only 10-20 pixels of vertical extent for something like a reference gauge board, which is insufficient for reliable pixel-level analysis.

The fix is **focal length**, not sensor megapixels. The recommended approach:

- **OV5647 (5MP sensor)** paired with a **manually adjustable or fixed telephoto M12-mount lens** (a well-documented modification in the Raspberry Pi camera community, since the OV5647 is the same sensor used in early Pi Camera modules) — this trades field of view for magnification, putting more pixels on the actual target
- Alternative: repurpose **CCTV/security-industry camera modules** that come with integrated optical zoom and long-range IR illuminators (rated 20-30m+), since long-range night vision at this distance is a solved problem in that industry, just not in the typical ESP32-CAM hobbyist ecosystem
- **Mandatory validation step**: physically test resolution-on-target at the real deployment distance and angle before finalizing a part — this cannot be correctly selected from a datasheet alone

Night/low-light operation requires either an IR-capable sensor (no IR-cut filter) paired with IR illumination that can actually project 20-30 meters (most cheap IR LED arrays only reach a few meters, so this needs a purpose-built long-range IR illuminator, again borrowed from the security-camera industry).

2.3 What the vision model actually computes — feature extraction, not classification

A critical design decision: **the on-node vision system does not output a final "flood: yes/no" judgment.** It outputs a small set of continuous numeric features per captured frame:

1. **Turbidity/color estimate** — a numeric proxy for how muddy/sediment-laden the water appears. Physical basis: upstream erosion and surface runoff increase suspended sediment concentration *before* the increased water volume itself has physically arrived downstream, since sediment-laden runoff can enter the channel from slopes closer to the observation point than the main flood pulse. This makes turbidity trend the **single strongest leading indicator** among the three extracted features.
2. **Turbulence score** — a rough measure of surface agitation/texture, computed from frame-to-frame variance or optical flow magnitude in the water region. Physical basis: increased flow rate/volume typically increases surface turbulence. Caveat: this signal is also affected by wind and normal daily flow variation unrelated to flooding, so it is a noisier, more ambiguous signal on its own and only becomes meaningful when compared against that specific site's own historical turbulence baseline.
3. **Water coverage percentage** — the fraction of the frame's expected river channel area that is currently covered by water, a coarse proxy for absolute water level. This is fundamentally a **confirmatory/detection signal, not predictive** — by the time coverage percentage rises, water has already arrived at that visual position.

None of these three features alone constitutes prediction. Prediction emerges from combining **rate of change over time** in these features with **cross-node correlation** (developed in Part 5) — this is the architectural point where "the intelligence lives in the network, not in any single node."

2.4 Fire/forest sensing

Primary approach: camera-based binary classification (fire/smoke present vs. absent), using a small convolutional neural network. The camera may be shared with the water-monitoring camera if a single wide frame covers both river and forest/slope (an open design question flagged for team decision), since fire/smoke visual detection is a well-precedented technique in real forest-fire camera networks — wide-area smoke/haze detection from elevated fixed cameras is an established real-world method.

Explicitly rejected as unreliable on its own: naive color-based fire detection ("classify orange pixels as fire") — this produces high false-positive rates from sunsets, orange-colored objects/tarps, and warning lights, which is precisely why a properly trained CNN with deliberately collected negative/false-positive training examples is required rather than a simple color heuristic.

Supplementary sensor: an IR flame sensor (specific part: **YG1006 flame sensor module**) — reframed not as a competing detection method, but as a **low-power, always-on wake trigger**. Since flame sensors draw negligible current and can run continuously, they can serve as a cheap pre-filter that wakes the power-hungry camera+AI pipeline only when there's a preliminary indication worth a closer look, saving battery.

2.5 Seismic/landslide/avalanche sensing — explicitly non-visual

This is the one hazard category where camera vision was deliberately excluded as the primary method, for a specific physical reason: **avalanches are frequently obscured by the whiteout/snow conditions they themselves create**, making optical sensing structurally unreliable exactly when it matters most. Landslide precursors (slow ground creep, crack widening) are also not inherently visual signals in the way flame or water-level change are.

Real early-warning systems for this hazard category (referenced from institutions like the Geological Survey of India and IIT Roorkee's landslide monitoring work) use:

- **Geophones** or seismic-grade accelerometers (specific upgrade path identified: **ADXL355 low-noise accelerometer**, versus the MPU6050 starting point) to detect ground micro-tremors preceding slope failure
- **Tilt sensors / inclinometers** (specific part: **SCA100T-D02** or generic dual-axis tilt sensor modules) to measure slope angle change over time
- **Extensometers** — sensors that measure the widening of surface cracks, a direct mechanical precursor to slope failure
- **Soil moisture sensors** (specific part: **capacitive soil moisture sensor v1.2**) — since soil saturation is a primary landslide trigger mechanism, tracking moisture trend is itself a predictive signal, analogous to turbidity for flood

This entire hazard category was flagged explicitly as the least-researched in the project, with a direct recommendation to consult ISRO-adjacent or geological-survey-experienced mentors for validated reference architectures rather than inventing an approach from first principles.

PART 3: HARDWARE — EXACT COMPONENTS AND WHY

3.1 The chip-selection logic, stated as a general principle

The recurring analytical error to avoid: assuming more powerful compute improves accuracy on a task that isn't compute-bound. Accuracy in water-level sensing, for example, is bounded by optics, calibration, and physical sensor placement — not by neural network capacity. Adding a more powerful AI chip to a task like this adds cost, power draw, and toolchain complexity for zero accuracy benefit. Compute should be matched to the *specific* computational demand of each task, not applied uniformly at the highest available tier "to be safe."

3.2 Microcontroller/SoC options evaluated

ESP32-S3 — the workhorse choice for lightweight sensing nodes (non-vision-heavy tasks, and orchestration/connectivity duties even on vision nodes). Rationale: mature ecosystem (Espressif's official ESP-IDF framework, and Arduino framework support), built-in AI vector instructions sufficient for genuinely simple TinyML tasks (wake-word style classification, coarse image classification with heavily quantized models), and well-documented low-power deep

sleep modes at the microamp level. Camera support available via modules like the **XIAO ESP32-S3 Sense**.

K210 (Kendryte K210) — a dual-core RISC-V 64-bit SoC (28nm process, IMAFDC instruction set, 8MB on-chip high-speed SRAM, 400MHz clock, overclockable to 800MHz) with a dedicated **KPU (Kendryte Processing Unit)** — a hardware neural network accelerator, not a general-purpose CPU running ML in software. The KPU is rated at 0.25 TOPS at 0.3W at 400MHz, or 0.5 TOPS at 800MHz overclocked, cited as sufficient for object recognition at 60fps at VGA resolution. This is the correct choice specifically for nodes running real CNN inference (fire/smoke classification, or future water-scene feature extraction via vision), because it performs this class of computation in dedicated silicon rather than software emulation on a general CPU — the same task on a Raspberry Pi Zero 2 W's CPU alone was benchmarked by community users at roughly 6 seconds per inference using standard TFLite object detection scripts, versus the K210's real-time capability.

K230 (Kendryte's successor to K210) — offers roughly 6 TOPS equivalent AI performance, cited as approximately 13.7 times the K210's AI throughput and 8.5 times its general CPU compute. Comes in modules with integrated support for 30+ pre-built AI vision features (object, face, gesture, text, digit recognition) and simplified training tooling. Trade-off: newer, less field-proven, smaller community/toolchain maturity than K210.

Sipeed Maixduino — a specific combined board (K210 + ESP32 module + integrated camera connector, in an Arduino Uno form factor) evaluated as a practical way to get both KPU-based inference and WiFi/Bluetooth connectivity on one PCB without designing a custom two-chip board from scratch. Specification details confirmed from its datasheet: power input via USB-C DC-DC step-down circuit supporting 6-12V input, providing 5V at 1.2A output; ESP32 module (specifically MSM261S4030H0-class) supports 2.4GHz 802.11 b/g/n WiFi up to 150Mbps and Bluetooth v4.2 (BR/EDR + BLE); 24-pin 0.5mm FPC camera connector. Important limitation identified: **this board's onboard wireless is WiFi/Bluetooth only — it does not include LoRa**, meaning an external LoRa module (e.g., an SX1276/RFM95-based module) must still be added via the board's exposed pins to fit the project's long-range mesh communication requirement.

Raspberry Pi Zero 2 W — evaluated and set aside for on-node use specifically because it lacks any dedicated neural network accelerator; it runs inference via its general-purpose quad-core Cortex-A53 CPU in software, which was benchmarked (via community-reported testing) at roughly 6 seconds per TFLite object-detection inference — acceptable for a task that only needs periodic checking (not real-time video), but a genuine step down in efficiency versus the K210's hardware-accelerated approach, and it draws meaningfully more idle/active power than an ESP32-S3 with no compensating advantage for this specific workload. It remains the correct choice for the **central hub**, not individual sensing nodes, once migrated from a laptop for field deployment.

3.3 The two-processor relationship on a combined board (K210 + ESP32)

A specific misconception addressed directly: on a board like the Maixduino, the K210 and ESP32 are **two entirely separate processors with their own separate RAM**, typically connected via UART or SPI — not a shared-memory architecture where one chip's memory is accessible to the other. The ESP32 cannot "help compute" the AI inference task; it has no path to meaningfully accelerate what happens inside K210's KPU.

The correct division of responsibility:

- **K210's job:** run the camera capture and KPU inference — the one task it's uniquely equipped for — and stay powered down otherwise
- **ESP32's job:** everything else — WiFi/BLE if needed, sleep/wake timing and power-sequencing logic, receiving the K210's inference result over UART, packaging that result into a LoRa packet, and handing it to the LoRa radio for transmission

This division is why, in the power design (Part 4), the ESP32 is the component trusted to manage deep sleep, while K210 is treated as a power-hungry peripheral that gets fully power-gated (switched off entirely, not merely "slept") between active bursts.

3.4 The modular mainboard + daughterboard architecture

The final hardware pattern adopted, connecting back to prior work on a general-purpose modular edge AI dev board: one **core mainboard** (compute + power management + LoRa comms — either an ESP32-S3-only "lite" tier, or an ESP32+K210 "vision" tier) paired with **swappable daughterboards** carrying only the hazard-specific sensor hardware, connected via a standardized pinout exposing I2C, a couple of ADC-capable analog pins, and power.

Water/flood daughterboard: camera module (primary) plus a tipping-bucket rain gauge (specific parts: **YL-83 rain sensor** for a cheap hobbyist version, or **Davis 7852 rain collector** for an industrial-grade, more accurate option) as a supplementary upstream-rainfall leading indicator that the camera structurally cannot capture.

Fire/forest daughterboard: camera module (primary, potentially shared with water sensing), plus the YG1006 IR flame sensor as a low-power wake trigger.

Seismic/landslide daughterboard: MPU6050 (baseline) or ADXL355 (upgrade path) accelerometer as primary, plus SCA100T-D02-class tilt sensor and capacitive soil moisture sensor v1.2 as supplementary precursor signals — this board is explicitly camera-free.

Shared across all daughterboard types: a simple resistor-divider circuit into an ADC pin for battery voltage monitoring (not a discrete purchasable sensor, but a required circuit on every variant, since battery health reporting is load-bearing for the power-degradation detection logic in Part 4).

An open engineering decision explicitly left to the team: whether to standardize on **one universal mainboard** (sized for the heaviest case, K210-class, used even on nodes that don't strictly need it — simpler to manufacture/test, but wastes cost and power on lighter-duty nodes)

versus **two mainboard tiers** (ESP32-S3-only "lite" and ESP32+K210 "vision," sharing the same daughterboard connector standard — more efficient per-node economics, at the cost of double the mainboard design and validation effort).

3.5 Physical/mechanical considerations

Camera lens maintenance was identified as a real, non-trivial field-reliability risk — a system that reads perfectly on a spec sheet but silently fails after the lens gets muddy, algae-covered, or obstructed by debris/insects/frost is not meaningfully "deployed." The layered mitigation strategy adopted, in priority order:

1. **Physical prevention** (free, mechanical): a downward-angled rain/sun hood shielding the lens from direct splash and precipitation, plus mounting position chosen to avoid normal splash zones
 2. **Self-detection of degraded readings**: firmware logic that checks image contrast/sharpness against an expected baseline and flags "sensor obstructed" as its own distinct status message, converting a silent failure into a known, actionable one — this is treated as more important than solving the cleaning problem itself, since visibility into a fault is what allows a response
 3. **Scheduled human maintenance visits** accepted as a normal, expected part of the operating model (analogous to how real weather stations and wildlife cameras are maintained), rather than treated as a design failure requiring elimination
 4. **Active mechanical cleaning mechanisms** (wipers, air-puff systems) explicitly deprioritized as unnecessary added complexity, failure points, and power draw for this project stage
-

PART 4: POWER SYSTEM — FULL DERIVATION

4.1 The governing design requirement: autonomy days, not average-case power

The critical reframe established: the correct sizing target is not "power the node on a typical day," but **"survive N consecutive days of near-zero solar input without failing."** This is the standard concept of **autonomy days** in off-grid solar system design. For a monsoon-heavy mountain deployment, extended overcast stretches of one to two weeks are a realistic design case, not a rare tail event, and the system must be sized against this explicitly.

4.2 State-based current draw model

Total energy consumption is modeled as the sum of time spent in each operating state multiplied by that state's current draw, rather than a single average figure — because the dominant driver of total power budget is **how aggressively the system can power down between active periods.**

States identified for a camera+K210 node:

- **Deep sleep:** target microamp-to-low-milliamp range, contingent on whether K210 can be fully power-gated (see 4.3)
- **Wake + camera initialization:** brief transitional current spike as the camera sensor rail powers up (roughly 80-150mA range, for 1-2 seconds)
- **Active capture + KPU inference:** the peak-draw state, roughly 200-300mA estimated, sustained for only 1-3 seconds per wake cycle
- **LoRa transmit:** roughly 120mA for a routine packet, momentarily higher during transmission bursts, sustained under 1 second

At a wake interval of every 5-15 minutes (900 seconds, using 15-minute cycles as the worked example), the node spends the overwhelming majority of each cycle — roughly 895 of 900 seconds — asleep, and only 3-5 seconds active. This means **average power draw is dominated by sleep-state current, not by the K210's active power appetite**, provided sleep is genuinely deep.

4.3 The critical unresolved variable: K210 sleep current

A key finding from direct research: **Kendryte's own public documentation does not state a verified, trustworthy sleep-mode or active-mode current figure in milliamps for the K210.** Marketing materials describe it generically as "low power," and the one concrete number found in official board documentation (a Sipeed MaixGo board recommendation of greater than 600mA supply current) is a board-level active-operation supply sizing recommendation — not a chip-level sleep-mode specification.

Because this number cannot be safely assumed, the adopted engineering default is: **do not rely on K210's own claimed sleep mode.** Instead, use the ESP32 (whose deep-sleep current is well-documented at the microamp level, a mature and trusted specification) to fully power-gate the K210 and camera rail using a MOSFET or dedicated load switch between the two chips' shared power rail and the battery. The ESP32 wakes on its own low-power hardware timer, switches power ON to the K210+camera circuit only for the duration of the active capture-and-inference burst, then cuts that rail's power entirely (not merely idling it) before returning to its own deep sleep. This makes the system's power budget independent of an unverified number: a fully power-gated K210 draws exactly zero current when its rail is switched off, regardless of what its internal "sleep mode" would have consumed if left powered.

This measurement is flagged as the single most important unresolved action item: physically measuring real current draw (active state, K210's own claimed sleep state, and fully power-gated/off state) using a multimeter or inline USB power meter on actual hardware, since this single measured value determines whether battery-sizing math holds as calculated or needs significant revision.

4.4 Worked daily energy budget (illustrative, pending real measurement)

Using estimated figures at 15-minute wake cycles (96 wake events per 24-hour day):

- Sleep contribution: approximately 23.9 hours at an estimated ~5mA average $\approx$ 120 mAh

- Active-burst contribution: $96 \text{ wakes} \times \sim 5 \text{ seconds average active duration} \times \sim 250 \text{mA}$
average current $\approx 96 \times 0.35 \text{ mAh} \approx 33 \text{ mAh}$
- **Total estimated daily energy requirement: approximately 150-180 mAh per day** (at the board's $\sim 5\text{V}$ regulated rail), equivalent to roughly $0.75\text{-}0.9 \text{ Wh/day}$

4.5 Autonomy-based battery sizing

Applying the 150-180 mAh/day figure ($\approx 0.9 \text{ Wh/day}$) against a target of surviving extended dark stretches:

- **7 days autonomy:** raw requirement $\approx 1,260 \text{ mAh}$; applying a realistic 1.5-2x safety margin for battery inefficiency, depth-of-discharge limits, and estimation uncertainty in the underlying current figures $\rightarrow$ practical target of **2,000-2,500 mAh**
- **10 days autonomy** (the explicitly stated worst-case planning target): raw requirement $\approx 150 \text{ Wh}$ over 10 days at 0.625W average power (using the mesh-always-listening scenario developed in 4.7) $\rightarrow$ at 12V nominal, this corresponds to roughly a **12.5 Ah battery**, with a recommended real-world target of **15-20 Ah** once margin is applied for cold-temperature capacity derating and estimation uncertainty
- **14 days autonomy:** roughly double the 7-day figure, in the **4,000-5,000 mAh** range at the lower voltage baseline, or proportionally scaled at 12V

4.6 Voltage architecture

The Maixduino board's own specification calls for **6-12V DC input**, internally regulated down to 5V at up to 1.2A via its own USB-C step-down circuit. This drove the recommendation of a **12V nominal battery system**, specifically **LiFePO4 chemistry** rather than standard Li-ion/LiPo, for the following reasons:

- Substantially better cold-temperature tolerance and rechargeability — directly relevant given likely cold nights/winter conditions at Himalayan-region deployment sites, where standard LiPo capacity and cycle life degrade meaningfully
- Much longer cycle life (typically $2,000\text{-}5,000+$ charge cycles versus roughly $500\text{-}1,000$ for standard Li-ion), relevant for a system intended to run unattended across multiple years without battery replacement
- Materially safer chemistry, with much lower thermal runaway/fire risk than standard LiPo — a meaningful consideration for unattended field hardware
- Trade-off accepted: LiFePO4 is physically larger and heavier per watt-hour of stored energy than LiPo, and typically costs more upfront

A **buck converter/step-down module** (specifically recommended: a $12\text{V-to-}5\text{V}$ or $12\text{V-to-}9\text{V}$ DC-DC buck converter module) was recommended as a safety buffer between a nominally- 12V LiFePO4 pack (which can read as high as $13\text{-}14.6\text{V}$ when fully charged) and the Maixduino's input, to guarantee the board never sees voltage above its rated maximum input even at full charge.

4.7 The mesh-always-listening scenario — a critical late-stage revision

A major reframe emerged from the requirement that LoRa mesh relaying requires nodes to forward other nodes' packets, meaning the **radio and its controlling MCU cannot be fully asleep on the same schedule as the sensing subsystem** — a sleeping relay node cannot receive and forward a packet passing through it.

This was resolved by explicitly separating two independent power domains within a single node:

Domain A — mesh relay function (ESP32 + LoRa radio): must remain continuously listening. Estimated continuous draw: ESP32 active (~80-160mA) + LoRa radio in receive/listen mode (~10-15mA) ≈ **roughly 112-130mA continuous**.

Domain B — sensing function (camera + K210): can remain on its own independent sleep/wake schedule (every 5-15 minutes), power-gated per section 4.3, regardless of Domain A's continuous operation.

Two scenarios were explicitly modeled and compared:

- **Scenario A (correct design):** only Domain A (radio/ESP32) stays continuously on; Domain B still sleeps normally → average draw approximately 115-130mA continuous, translating to roughly 5.6 days of runtime on a 7Ah battery, or roughly 16 days on a 20Ah battery, with zero solar input
- **Scenario B (rejected design):** everything, including camera and K210, held continuously active with no sleep at all → average draw balloons to approximately 440mA continuous, translating to only about 1.6 days of runtime on a 7Ah battery, or roughly 4.5 days on a 20Ah battery — incompatible with any multi-week unattended operation goal regardless of how large a battery is used

Explicit recommendation: never disable sleep for the entire node. The mesh-relay radio function must stay listening continuously as a hardware/protocol necessity, but the camera/K210 sensing subsystem should retain its independent sleep schedule regardless, since there is no networking requirement forcing it to run continuously. This hybrid approach is what keeps a 12V solar+battery system viable at all.

4.8 Contingency and graceful degradation beyond fixed battery sizing

Beyond static battery capacity, three additional resilience layers were identified:

Adaptive wake-interval throttling: firmware logic where the node monitors its own battery voltage (already required for the standard telemetry packet) and automatically extends its wake interval — for example, stretching from a normal 10-minute cycle to 30-60 minutes — once voltage drops below a defined threshold, trading responsiveness for extended survival specifically during an extended dark stretch, allowing the system to degrade gracefully rather than fail outright. This logic mirrors, at the node level, the same battery-voltage field already transmitted to the hub for network-wide monitoring.

Hybrid/complementary generation sources (proposed as forward-looking, differentiated design, not required for the initial build):

- **Wind:** compact off-grid wind turbines suited to remote-sensor scale, justified by the observation that overcast conditions and low wind don't necessarily correlate — a cloudy but windy stretch could still provide charging via a combined solar+wind charge controller
- **Micro-hydro:** given the nodes' inherent proximity to flowing water, small in-stream micro-hydro generators (designed for low-flow-rate, small-scale off-grid sensor applications) could provide charging largely independent of weather, since river flow tends to be far more consistent than sunlight in a monsoon-adjacent region — explicitly flagged as a differentiated pitch point specific to this project's deployment context
- **Primary (non-rechargeable) backup cells:** a small industrial lithium primary cell (e.g., LiSOCl₂ chemistry, as used in remote utility meters for extremely long shelf life and low self-discharge) wired as a last-resort failsafe that only engages if the main rechargeable battery drops below a critical voltage floor

4.9 Recommended concrete components (for sourcing)

- **Battery:** 12V LiFePO₄ pack, 15-20Ah capacity — search terms: "12V LiFePO₄ battery 15Ah/20Ah," brands referenced: Redway, Renogy, Zeus, Loom Solar (India-based)
- **Charge controller:** a controller explicitly rated for LiFePO₄'s charge voltage profile (not all controllers support this chemistry correctly) — search: "12V solar charge controller LiFePO₄," brands referenced: Victron (higher-end), EPEVER, Renogy; MPPT preferred over PWM for 5-20% better charging efficiency, particularly valuable given cloudy/low-light mountain conditions
- **Solar panel:** monocrystalline, 10-20W, 12V nominal output — search: "12V 10W/20W monocrystalline solar panel," brands referenced: Loom Solar, Luminous, Waaree; must be explicitly rated IP65 or higher for weather exposure, with an aluminum frame (for secure pole-mounting against wind) and a pre-wired junction box (not bare terminals)
- **Buck converter:** search: "DC-DC buck converter 12V to 5V/9V 3A" — as the voltage-safety buffer described in 4.6

For the immediate 3-day hackathon demo specifically, a simpler standard 12V 7Ah sealed lead-acid or basic Li-ion pack (readily available same-day) was recommended as sufficient for demonstration purposes, with the full LiFePO₄/MPPT/monocrystalline specification explicitly documented as the researched production-deployment plan rather than something that needed to be sourced and tested within the demo timeframe.

PART 5: THE PREDICTION ALGORITHM — FULL LOGIC

5.1 Where each computation happens, and why

A foundational architectural decision: **individual nodes do not retain their own historical data or compute their own baseline**. This was deliberately settled after examining the memory/persistence problem directly — if a node's K210 is power-gated off between cycles (per Part 4.3), any data sitting in its RAM is lost the instant power is cut, meaning cross-cycle statistical memory cannot live in node RAM at all.

Rather than solving this with local flash/SD-based persistence on each node (a viable but more fragile alternative — subject to flash wear, physical failure, or loss if a node is replaced), the design deliberately centralizes all historical memory and cross-node reasoning at the hub, for these reasons:

- The hub never loses power the way a sleeping/power-gated node does
- Historical memory shouldn't be tied to a single physical node's local storage surviving intact, especially given the OTA re-flashing capability that might involve replacing node hardware
- Cross-node correlation is *structurally impossible* to compute on any single node, since no individual node has visibility into any other node's readings — this reasoning can only happen at a point that receives data from multiple sources, which is the hub by definition

This means each node's actual job, restated precisely: **wake, read sensor(s), compute the current frame's features (turbidity, turbulence, coverage, or accelerometer/tilt/moisture readings, depending on hazard type), transmit those raw current-cycle values, and return to sleep**. No local statistics, no local history, no local decision-making about whether to alert.

5.2 The rolling baseline — mathematical formulation

For each node, the hub maintains a rolling window of that node's historical readings (recommended trailing window: on the order of 30 days, chosen to be long enough to smooth out daily noise while still adapting to genuine seasonal drift, such as the transition into monsoon season).

For each feature type (e.g., turbidity), the hub computes:

- **Rolling mean (μ)**: the average value of that feature over the trailing window
- **Rolling standard deviation (σ)**: the spread/variability of that feature over the same window

A new incoming reading is then evaluated not against a fixed number, but against a **z-score-style deviation measure**: how many standard deviations away from this specific node's own recent mean does the new reading fall? A commonly cited example threshold discussed was flagging deviation when a reading falls **3 or more standard deviations** above the rolling mean — a statistically significant departure from that site's own established normal, regardless of what that site's absolute baseline numbers happen to be.

This is what allows the exact same escalation logic to correctly handle a gently flowing lowland river and a fast mountain torrent without any manual per-site calibration: each node's "abnormal" is defined relative to its own accumulated history, not a shared constant.

5.3 Rate of change as the primary signal, not absolute value

A specific refinement established during the design process: **rate of change is a more portable and more genuinely predictive signal across different rivers than absolute feature values**. A flood surge produces an unusually fast *rise* relative to that river's own typical behavior, regardless of whether that river's baseline sits at a low or high absolute value. This makes rate-of-change deviation (the first derivative of a feature over time, compared against that node's historical rate-of-change distribution) the primary quantity the escalation logic watches, with absolute-value deviation serving a secondary, confirmatory role.

5.4 Cross-node correlation and travel-time estimation

This is identified as the actual mechanism that produces genuine predictive lead time, and it is explicitly a network-level property that no single node's sophistication can substitute for.

The logic: when an upstream node's reading deviates significantly from its own baseline (per 5.2), the hub does not immediately escalate to a full alert. Instead, it checks whether the **same relative deviation pattern subsequently appears at the next node downstream**, within a time window calculated from the known physical distance between the two nodes and a reasonable estimate of the river's typical flow speed at that reach.

This travel-time estimate is basic kinematics: **time = distance ÷ velocity**. The distance between any two nodes is known precisely because of the geotagging mechanism (Part 7): each node's physical GPS coordinate is captured at deployment time and stored against its unique node ID, allowing the hub to compute real inter-node distances directly. The flow-velocity estimate would be refined over time using the project's own accumulated dataset (correlating how long it actually took a known event to propagate between two specific nodes in the past), making the propagation-time estimate itself improve as more real seasonal data accumulates — directly tying back to the project's dataset-building value.

If the expected downstream deviation is confirmed within the calculated travel-time window, confidence in a genuine, propagating flood event increases substantially, and the hub can issue an alert to communities further downstream **with a stated estimated arrival time** — this is the concrete mechanism that produces actionable lead time (potentially hours), rather than the single-point, immediate-only warning a detector-style system would give.

5.5 Why the escalation logic is rule-based, not a black-box learned model

An explicit design principle: given that this is a life-safety system where false negatives (missed real floods) carry severe consequences, the escalation decision logic (baseline comparison, rate-of-change thresholding, cross-node correlation, travel-time estimation) is implemented as **transparent, auditable, rule-based logic** rather than an opaque trained model making the final

call. This is consistent with how real operational flood-forecasting systems (referenced: India's Central Water Commission) function at the operational level — level, rate-of-change, and upstream correlation logic, rather than deep learned prediction — specifically because such systems must be explainable and every decision must be traceable to a specific, inspectable cause.

A learned predictive layer was explicitly discussed as a legitimate **future enhancement**, to be layered on top of the rule-based system once a real season or more of accumulated historical data exists — but positioned as an addition that refines the rule-based system's judgment, not a replacement for it, preserving auditability.

PART 6: MACHINE LEARNING PIPELINE — FULL DETAIL

6.1 The distinction between "the model" and "the firmware," clarified precisely

A key conceptual clarification made explicit: a trained neural network model is not, by itself, a runnable program. It is a set of learned numeric weights plus an architecture definition. Running it on a K210 in the field requires a separate firmware program (written in MicroPython via **MaixPy**, or in C using Sipeed's official SDK) that handles everything *around* the model: waking from sleep, initializing the camera, capturing a frame, invoking the model as a single function call (loading the compiled model file and feeding it the captured frame), interpreting the resulting confidence score against a decision threshold, packaging the result, and handing it off for transmission before returning to sleep. The model is one function call inside a larger firmware program — not the entire program.

6.2 The full pipeline, step by step

Step 1 — Task definition: define the exact classification task as narrowly as useful. For fire detection, the recommended starting scope is strictly binary — "fire/smoke present" versus "normal scene" — deliberately avoiding a more complex multi-class scheme (fire vs. smoke vs. haze vs. clear) until the simpler binary version is validated, since a narrower task requires less training data and is easier to evaluate correctly. This step also requires explicitly defining hard negative cases in advance — sunsets, orange-colored objects, vehicle lights — the deliberate false-positive traps that a naive color-based approach would fail on.

Step 2 — Dataset collection and labeling: the identified bottleneck of the entire pipeline, more time-consuming than any subsequent step. Public datasets identified as strong starting points:

- **Roboflow's Fire Detection dataset** — approximately 9,000 annotated images specifically distinguishing active flames and smoke plumes from non-threatening environmental factors like sunsets and orange objects, directly addressing the false-positive concern from Step 1

- **The Flare Guard GitHub project** (repository: [sayedgamal99/Real-Time-Smoke-Fire-Detection-YOLO11](#)) — 10,463 annotated images plus a complete working reference pipeline (training, detection, and alerting via messaging integration) usable as a structural template even though it targets YOLO11/GPU deployment rather than K210 directly
- For flood/water: a **GitHub repository (duchieu260503/Flood-detection)** implementing YOLOv8 segmentation to detect water extent, plus the actual mathematics to convert detected pixel position to real-world water level using a physical reference stick visible in frame with two known calibration points — identified as the closest existing reference to the project's own gauge-reading concept
- The **FloodIMG dataset** (published research, 9,200+ images with bounding boxes and severity/depth scoring) and the **RIWA river water segmentation dataset** as additional flood-specific resources
- Explicit finding: **no pre-trained, ready-to-deploy K210-format fire or flood detection model exists publicly** — what exists is mature *tooling* for training your own quickly, not a plug-and-play solution, because these are narrow, deployment-specific tasks where a model trained on someone else's camera angle and lighting conditions transfers poorly to a new site (the same underlying reasoning that motivates the per-site baseline-learning approach in Part 5, applied here to vision model generalization)
- Labeling tools referenced: **Labellmg** or **Roboflow's** own labeling interface

Step 3 — Dataset splitting: standard machine learning practice of dividing labeled data into training (approximately 70%), validation (approximately 15%), and test (approximately 15%) subsets — training data is what the model learns from, validation data checks progress during development without contaminating the final evaluation, and test data is held back completely until the very end to produce an honest, unbiased accuracy figure.

Step 4 — Architecture selection: use a proven, lightweight, K210-compatible architecture rather than designing one from scratch or reaching for a large/exotic model. Specifically recommended: **MobileNetV1 or MobileNetV2**, or a small custom CNN, chosen because Kendryte's **nncase** conversion toolchain has known, well-tested support for these architecture families — an exotic or very recent architecture risks failing the later conversion step entirely, wasting significant time. **Transfer learning** (starting from ImageNet-pretrained weights and fine-tuning only on the project's own dataset) is recommended over training from randomly initialized weights, since it requires substantially less data and training time to reach good accuracy.

Step 5 — Training: implemented in Python using **TensorFlow/Keras** (preferred over a PyTorch-based path for this specific hardware target, since nncase's TensorFlow Lite input support is more mature than its ONNX/PyTorch path). Training script structure: load labeled images, apply data augmentation (random brightness, rotation, and cropping to improve robustness to real-world lighting/framing variation), then fine-tune the pretrained MobileNet-class model on the project's labeled dataset. This step runs entirely on a standard PC/laptop with a GPU and has no direct connection to the K210 yet — it is conventional ML training.

Step 6 — Honest evaluation: assess the trained model against the held-out test set, with explicit attention to the confusion matrix, not just overall accuracy — specifically the **false-negative rate** (real fires/floods missed) versus the **false-positive rate** (false alarms). Given this is a life-safety system, the design principle established is that missed real events are categorically worse than false alarms, which may justify deliberately shifting the decision threshold to favor catching more true positives even at the cost of more false alarms, with this tradeoff explicitly verified against the test set rather than assumed.

Step 7 — Quantization and conversion for K210: the trained Keras model is first converted to TensorFlow Lite format, then processed through **nncase** (Kendryte's official model conversion toolchain), which quantizes the model (reducing numeric precision from 32-bit floating point to 8-bit integers, a requirement of the KPU hardware) and compiles it into the K210-native **.kmodel** file format. This step is explicitly flagged as prone to needing iteration, since quantization can noticeably reduce accuracy — the recommended practice is to re-test the *quantized* model's accuracy against the test set again after conversion, not to trust the pre-quantization accuracy figure as final. A training framework called **aXeleRate** was identified as a tool that automates much of this training-to-conversion pipeline specifically for K210 targets, including a documented reference tutorial (a "raccoon detector" project) demonstrating the full flow from labeled images through automatic .kmodel conversion and even streaming inference results over UART — directly analogous to the project's own node-to-ESP32 handoff pattern.

Step 8 — Deployment and real-hardware validation: load the final .kmodel onto the K210 via the node's actual firmware, and test using the real camera in its real mounting position — since lab-tested performance often diverges from field performance due to the specific lens characteristics, lighting conditions, and image artifacts unique to the deployed hardware. This is also where firmware-level decision logic is tuned and validated — for instance, requiring multiple consecutive positive detections across successive frames before triggering an alert, to guard against a single-frame false positive triggering an unnecessary escalation.

6.3 GPU training time estimates (RTX 3050 vs RTX 4090)

For the roughly 9,000-image fire dataset, using transfer learning on a MobileNet-class architecture:

- **RTX 3050 (6GB VRAM):** raw training time of roughly **30-90 minutes per run** for approximately 20-30 epochs, with realistic total hands-on iteration time (including 2-4 rounds of adjustment and retraining based on evaluation results) spanning roughly **half a day to a full day**
- **RTX 4090 (24GB VRAM):** raw training time compressed to roughly **5-15 minutes per run**, a 5-8x speedup from dramatically higher CUDA core count, memory bandwidth, and available VRAM allowing larger batch sizes; total realistic hands-on iteration time compressed to roughly **2-4 hours**
- **Critical caveat established in both cases:** GPU choice has no effect on dataset collection/labeling time, the human-judgment-driven evaluation and threshold-tuning

loop, the CPU-bound nncase quantization/conversion step, or real-hardware validation on actual K210 hardware — these remain the dominant time costs regardless of training GPU, meaning a faster GPU compresses iteration speed but does not proportionally compress the project's overall timeline

PART 7: COMMUNICATION — LORA MESH IN DETAIL

7.1 Why LoRa, and why not alternatives

Cellular and WiFi connectivity is unreliable to entirely absent in mountain and river-valley terrain, ruling out both as primary long-range links. **LoRa (Long Range radio)** was selected specifically because it provides long-range communication (kilometer-scale under good line-of-sight conditions) at very low power draw, operating in a license-free ISM radio band, making it suitable for battery/solar-powered nodes with no ongoing connectivity cost.

7.2 Frequency band selection

433 MHz (a sub-GHz band, license-free in India) was selected over the 868/915 MHz LoRa variants specifically because lower-frequency radio waves penetrate terrain features and foliage more effectively — directly relevant given the mountain valley and forest deployment context, where higher-frequency signals would suffer greater obstruction loss.

7.3 Mesh topology, not star topology

A star topology (every node communicating directly to one central gateway) was explicitly rejected in favor of a **multi-hop mesh**, because mountain terrain (valleys, ridgelines) will routinely break direct line-of-sight between a distant node and a single fixed gateway location. In a mesh, nodes relay packets through intermediate neighboring nodes toward the gateway, tolerating the loss of any single direct path. Dedicated **relay-only repeater nodes** (carrying no sensors at all, just a LoRa radio, ESP32, and solar/battery power) were proposed as a solution for geographic gaps where sensing nodes alone don't provide adequate mesh connectivity, and elevated/ridge-advantaged mounting positions were noted as dramatically extending usable LoRa range even with a small elevation gain.

7.4 Message format specification

Given LoRa's inherently small payload capacity (typically in the range of 50-250 bytes depending on the spreading factor configuration used), the message format is a **compact fixed-layout binary structure**, explicitly not JSON or any text-based/self-describing format, to minimize airtime and power cost per transmission:

```
[node_id: 1 byte][timestamp: 4 bytes][msg_type: 1 byte]
[primary_reading: 2 bytes][rate_of_change: 2 bytes]
[battery_mv: 2 bytes][secondary_flag: 1 byte][checksum: 1 byte]
```

Total: approximately 14 bytes per packet. The **msg_type** field distinguishes at minimum between routine status updates (transmitted on a fixed schedule, e.g., every 5-15 minutes) and priority alerts (transmitted immediately upon detection of a significant deviation, outside the normal schedule) — allowing the hub's software to prioritize processing of alert-type packets. A **checksum byte** is included for basic data integrity verification given the inherent unreliability of long-range radio links. Raw high-resolution data — full camera frames, extended sensor logs — is explicitly never transmitted over LoRa due to payload size constraints; this data remains local to the node (stored on SD card) and is retrieved manually or during scheduled maintenance visits, serving the project's separate long-term dataset-building goal.

PART 8: THE CENTRAL HUB — SOFTWARE ARCHITECTURE

8.1 Hardware progression

Development/prototyping phase: a standard laptop, chosen deliberately over a Raspberry Pi at this stage specifically for faster iteration — full Python tooling (pandas, scikit-learn, standard debugging environments) without the friction of developing on constrained embedded hardware.

Field deployment phase: migration to a **Raspberry Pi** (4 or 5) once the correlation and escalation logic has been validated, chosen for its ability to run unattended for extended periods with modest power draw, appropriate for a system that itself needs solar/battery power at a field-deployed gateway location.

8.2 Software responsibilities, in full

The hub is the only component in the entire system with persistent, cross-node, cross-time memory, and its responsibilities are correspondingly the parts of the system that are structurally impossible to push down to individual nodes:

1. **Packet ingestion and parsing:** receiving raw binary LoRa packets via a gateway radio module (referenced specific parts: SX1276 or RFM95-based LoRa modules connected via UART/SPI to the Pi), decoding the fixed binary structure from Part 7.4 into structured data
2. **Persistent historical storage:** every parsed reading, from every node, is logged with a timestamp into a database (specifically referenced: **SQLite** for structured queryable storage, or plain **CSV** for simpler logging) — this accumulating store serves double duty as both the operational memory needed for baseline computation (Part 5.2) and as the project's long-term research dataset asset
3. **Rolling baseline computation** (Part 5.2): maintained per node, per feature type, continuously updated as new readings arrive
4. **Cross-node correlation and travel-time estimation** (Part 5.4): the core predictive computation, requiring the full multi-node dataset described above

5. **Escalation decision logic** (Part 5.5): the final rule-based determination of whether, and how urgently, to issue an alert
6. **Alert dispatch**: triggering the SMS notification pipeline (Part 9)
7. **Node health monitoring**: tracking battery voltage trends per node (from the telemetry field already present in every packet) to flag nodes showing sustained decline as an operational maintenance signal, distinct from the hazard-detection alerting pipeline

8.3 Why the hub, not the node, despite "most processing happening on the node"

Directly addressing the apparent tension: even with substantial feature-extraction computation happening on-node (turbidity/turbulence/coverage calculation, or accelerometer signal processing), the hub's role is not diminished, because its function is categorically different from a node's. Nodes compute a **single current reading**. The hub's entire value is built from **holding history across time and correlating across independent physical locations** — neither of which a resource-constrained, intermittently-powered, single-location node can do by itself, regardless of how much on-node compute capability it has. The Raspberry Pi's suitability for this role comes not from raw inference throughput (which it comparatively lacks versus a K210's dedicated KPU) but from its capacity for persistent storage, multi-source data fusion, and continuous operation, which are the genuinely hub-specific requirements.

PART 9: ALERTING MECHANISM — SMS DISPATCH

9.1 Two fundamentally different technical approaches

Cell broadcast (mass alert to all phones in a geographic area, by tower coverage, not by known phone number): this is how real government-level emergency alerts function, implemented in India through the **NDMA's Common Alerting Protocol (CAP), delivered via the Integrated Alert System / SACHET platform**, working through direct telecom operator integration. This approach was explicitly identified as **not independently buildable** within this project's scope, since it requires authorized government/disaster-authority status and direct telecom operator agreements — appropriately documented in project materials as "designed to integrate with NDMA's SACHET/Cell Broadcast system" as a future-scope/integration target, rather than a built feature.

Registered-number SMS dispatch (buildable now): a database mapping specific phone numbers to specific geographic zones or node coverage areas, built through a registration process (community sign-up, local administration coordination). When the hub's escalation logic fires an alert tied to a specific node, it queries the registration database for numbers associated with that node's zone (and, per the travel-time model in Part 5.4, potentially the zones of downstream nodes within the estimated arrival window), then dispatches SMS to that specific, known list.

9.2 Physical SMS dispatch mechanism

For actual message transmission, two options were evaluated:

- **A GSM module with a local SIM card** (specific parts referenced: **SIM800L** or **SIM7600**) connected directly to the hub hardware — sends SMS directly to the cellular network without requiring internet connectivity at the hub, which aligns well with the project's low-connectivity mountain deployment assumption. Noted limitation: sends messages roughly one at a time with several seconds per message, adequate for hundreds of recipients but comparatively slow for very large recipient lists
- **A cloud SMS API service** (referenced: Twilio, MSG91, Fast2SMS for the Indian market) — offers much faster bulk sending, but requires the hub itself to have internet connectivity, which may not hold if the hub is deployed at the same remote, low-connectivity location as the sensor nodes it serves

Recommendation: the GSM-module approach is more architecturally consistent with the project's mountain-deployment constraints, particularly if the hub sits at a location with cellular signal even where individual sensor nodes do not.

9.3 Zone-based versus dynamic geofencing for recipient selection

The simplest implementable version assigns each registered phone number to its nearest node/zone at registration time, with an alert for that node (or nodes calculated to be upstream within the propagation model) triggering SMS to everyone registered in that zone. A more sophisticated future-scope version would use registered precise GPS coordinates per number and dynamically calculate real distance against the live flood-propagation estimate to decide recipients in real time — explicitly noted as adding meaningful complexity (geofencing computation, ongoing user location data requirements, associated privacy considerations) beyond the zone-based version's scope.

PART 10: OPERATIONAL/DEPLOYMENT LAYER

10.1 Remote firmware updates via a companion phone app

A mobile app connects to a deployed node via USB (using Android's USB-OTG/serial capability), replicating the function of standard desktop flashing tools for each chip family (an esptool-equivalent flow for ESP32, a kflash-equivalent flow for K210). The app downloads the correct firmware binary — tagged by node type and version (e.g., a naming convention like flood-v1.2.bin) — from a cloud server, flashes it to the connected device, and critically, **verifies successful boot and correct version reporting before marking the update as successful**, specifically to avoid a scenario where a field-flashed device is left bricked with no easy recovery path, given the practical difficulty of a return visit to a remote mountain site.

10.2 Node geotagging at deployment

At the moment of physical installation (explicitly distinguished from the moment of firmware flashing/preparation, which may happen earlier at a workshop, away from the final site), the app captures the phone's own GPS coordinate — not a GPS module on the node itself, since duplicating GPS hardware on every node would add unnecessary cost and power draw when the deploying technician's phone already provides an equivalent one-time coordinate — and pushes this location to a cloud database (referenced: **AWS**), tagged against that specific node's unique hardware ID.

This stored node-ID-to-location mapping is what allows the central hub to compute real physical inter-node distances, which is a direct input to the travel-time propagation estimate described in Part 5.4 — without accurate geotagging, the hub's correlation logic would have no basis for estimating how long a detected upstream event should take to reach a downstream node.

PART 11: FUTURE-SCOPE ITEMS EXPLICITLY DOCUMENTED (NOT BUILT)

For completeness and honesty in project framing, the following were identified as legitimate, researched extensions explicitly scoped as future work rather than components of the current build:

- A **tiered dashboard product** for government/disaster-authority officials, structured as a free base tier (live node status, real-time alerts, basic map view, historical alert log — deliberately kept free and unconditional since gating actual life-safety alerts behind payment was identified as both ethically wrong and reputationally damaging) plus a paid premium tier (trend/seasonal analytics, cross-node correlation visualizations, multi-site administrative management, custom threshold tuning, data export/API access, integration hooks into existing government emergency broadcast systems)
- **Cell-broadcast/NDMA SACHET integration** for true mass geographic alerting, as distinct from the registered-number SMS approach that is independently buildable
- **Wind and micro-hydro supplementary power generation**, as detailed in Part 4.8
- **Two-tier mainboard standardization** (lite ESP32-S3 versus full K210 vision tier) as an open cost/engineering-effort decision
- A **learned predictive model layered atop the rule-based escalation logic**, to be developed once sufficient real seasonal historical data has accumulated through the system's own operation